

Guia de Estudo de Python

1. Fundamentos

O que é Python e suas aplicações

Variáveis e tipos: int, float, str, bool

Entrada e saída de dados (input e print)

Operadores aritméticos, lógicos e relacionais

2. Estruturas Condicionais

if, elif, else

Operadores lógicos (and, or, not)

Condições aninhadas

3. Estruturas de Repetição

while

for

range()

break e continue

4. Estruturas de Dados

Listas (append, pop, sort)

Tuplas

Dicionários (chave e valor)

Sets (conjuntos)

7. Tratamento de Erros

try, except

finally

Tipos de exceções

8. Arquivos

Leitura e escrita com open()

Modos r, w, a

Contexto with

9. Programação Orientada a Objetos

Classes e objetos

__init__

Encapsulamento

Herança

Polimorfismo

10. Módulos e Pacotes

import

Criação de módulos próprios

Bibliotecas padrão (math, datetime)

5. Funções

Definição com def

Parâmetros e retorno

Escopo local e global

Funções lambda

6. Manipulação de Strings

Métodos: lower, upper, split, replace

Formatação com f-strings

len()

Quando usar:

Preciso guardar vários dados? → lista

Preciso repetir algo? → loop

Preciso decidir algo? → if

Preciso organizar código? → função

O usuário pode errar? → validação

11. Conceitos Intermediários

List Comprehension

Geradores (yield)

Decoradores

Expressões regulares (re)

12. Avançado

Programação funcional (map, filter, reduce)

async e await

Threads e Multiprocessing

Testes automatizados (unittest, pytest)

APIs REST

Banco de dados (SQLite, PostgreSQL, MySQL)

ORM (SQLAlchemy)

Docker e boas práticas

Complexidade Big-O

PYTHON

— VARIÁVEIS

— CONDIÇÕES

— LOOPS

— ESTRUTURAS DE DADOS

— LISTA

— TUPLA

— DICIONÁRIO

— FUNÇÕES

PROBLEMA

- Preciso guardar dados?
 - Um valor → variável
 - Muitos valores → lista
 - Dados fixos → tupla
 - Dados com nome → dicionário
- Preciso tomar decisão?
 - if / elif / else
- Preciso repetir algo?
 - Quantidade definida → for
 - Até acontecer algo → while
- Código está repetindo?
 - criar função
- Usuário pode digitar errado?
 - validação

PYTHON

- TIPOS DE DADOS
 - int
 - números inteiros
 - exemplos: 10, 25, -3
 - float
 - números decimais
 - exemplos: 3.5, 7.8
 - str (string)
 - textos
 - exemplos: "Python", "Olá"
 - bool
 - valores lógicos
 - True (verdadeiro)
 - False (falso)
- FUNÇÕES
 - O que são
 - blocos de código reutilizáveis
 - organizam o programa
 - Estrutura
 - def nome_da_funcao():
código
 - Uso
 - repetir tarefas
 - dividir partes do programa
- PARÂMETROS
 - |
 - |

—	Dados que entram na função
—	Exemplo
	def soma(a, b): print(a + b)
—	Chamada
	soma(5, 3)
—	RETURN
—	Devolve um valor da função
—	Estrutura
	def soma(a, b): return a + b
—	Uso
	resultado = soma(5,3)
—	VARIÁVEL = 0
—	Usado para iniciar contagem ou soma
—	Exemplos
	soma = 0 contador = 0
—	Usado em
	• loops • acumuladores
—	TRUE E FALSE

—	Valores lógicos
—	Usados em
	• condições • verificações
—	Exemplo
	idade = 18 print(idade >= 18)
—	Resultado
	• True ou False
—	FUNÇÕES COM STRING
—	Trabalham com texto
—	Exemplo
	def saudacao(nome): print("Olá", nome)
—	Uso
	saudacao("Nayra")
—	FUNÇÕES COM NÚMEROS
—	Realizam cálculos
—	Exemplo
	def multiplicar(a, b): return a * b
—	Uso

```
# Cores no Python (ANSI Escape Code)
# \033[0;33;44m MODELO EXEMPLO
# 0 = STYLE 33 = TEXT 44 = BACKGROUND
#
```

```
-----
--
```

# STYLE	TEXT	BACKGROUND
# 0 - Nenhum	30 - Branco	40 - Branco
# 1 - Bold	31 - Vermelho	41 - vermelho
# 4 - Sublinhado	32 - Verde	42 - Verde
# 7 - Negativo	33 - Amarelo	43 - Amarelo
#	34 - Azul	44 - Azul
#	35 - Majenta	45 - Majenta
#	36 - Ciano	46 - Ciano
#	37 - Cinza	47 - Cinza

```
#-----
-----
```

```
# Importante: Sempre coloque \033[m no final para resetar a
cor,
# senão o terminal continua colorido.
```

— 1. FUNDAMENTOS

- |
- | — Variáveis
- | | — int
- | | — float
- | | — str
- | | — bool
- |
- | — Entrada e saída
- | | — input()
- | | — print()
- |
- | — Operadores
- | | — aritméticos (+ - * / // % **)
- | | — relacionais (== != > < >= <=)
- | | — lógicos (and, or, not)
- |
- | — Conversão de tipos
- | | — int()
- | | — float()
- | | — str()
- |
- |

— 2. CONTROLE DE FLUXO

- |
- | — Condicionais
- | | — if
- | | — elif
- | | — else
- |
- | — Operadores lógicos
- | | — and
- | | — or
- | | — not
- |
- | — Condições aninhadas

— 3. ESTRUTURAS DE REPETIÇÃO

- |
- | — while
- | | — repetição por condição
- |
- | — for
- | | — percorrer sequência
- |
- | — range()
- | | — range(n)
- | | — range(inicio,fim)
- | | — range(inicio,fim,passo)
- |
- | — Controle de loops
- | | — break

- | | ↳ continue
- | |
- | ↳ Loops aninhados
- |
- |
- | — 4. ESTRUTURAS DE DADOS
- |
- | ↳ Listas
- | | ↳ append()
- | | ↳ insert()
- | | ↳ pop()
- | | ↳ remove()
- | | ↳ sort()
- | | ↳ reverse()
- | |
- | ↳ Tuplas
- | | ↳ imutáveis
- | | ↳ indexação
- | |
- | ↳ Dicionários
- | | ↳ chave → valor
- | | ↳ keys()
- | | ↳ values()
- | | ↳ items()
- | |
- | ↳ Sets
- | ↳ conjuntos
- | ↳ sem repetição
- |

- |
- | — 5. FUNÇÕES
- |
- | ↳ definição
- | | ↳ def
- | |
- | ↳ parâmetros
- | | ↳ posicionais
- | | ↳ nomeados
- | | ↳ padrão
- | |
- | ↳ retorno
- | | ↳ return
- | |
- | ↳ escopo
- | | ↳ local
- | | ↳ global
- | |
- | ↳ lambda
- | | ↳ funções anônimas
- | |
- | ↳ recursão
- |
- | — 6. STRINGS
- |
- | ↳ métodos
- | | ↳ lower()
- | | ↳ upper()
- | | ↳ split()
- | | ↳ replace()
- | | ↳ strip()

		— formatação
		f-string
		format()
		%
		— manipulação
		slicing
		len()
		— 7. TRATAMENTO DE ERROS
		try
		except
		else
		finally
		tipos de erro
		ValueError
		TypeError
		ZeroDivisionError
		IndexError

		— 8. ARQUIVOS
		open()
		r → leitura
		w → escrita
		a → adicionar
		read()
		write()
		with open()
		— 9. PROGRAMAÇÃO
		ORIENTADA A OBJETOS
		classes
		objetos
		construtor
		init
		conceitos
		encapsulamento
		herança
		polimorfismo
		métodos

— 10. MÓDULOS E PACOTES

- | | **import**
- | | **from module import**
- | |
- | | **bibliotecas padrão**
- | | | **math**
- | | | **datetime**
- | | | **random**
- | | | **os**
- | |
- | | **módulos próprios**

— 11. CONCEITOS INTERMEDIÁRIOS

- | | **list comprehension**
- | | **generator (yield)**
- | | **decorators**
- | | **expressões regulares (re)**
- | | **iteradores**

— 12. PROGRAMAÇÃO FUNCIONAL

- | | **map()**
- | | **filter()**
- | | **reduce()**

— 13. CONCORRÊNCIA

- | | **threading**
- | | **multiprocessing**
- | | **async / await**

— 14. BANCO DE DADOS

- | | **SQLite**
- | | **PostgreSQL**
- | | **MySQL**
- | |
- | | **ORM**
- | | **SQLAlchemy**

— 15. DESENVOLVIMENTO WEB

- | | **Flask**
- | | **Django**
- | | **FastAPI**

— 16. APIs

- | | **REST**
- | | **JSON**
- | | **requests**

└─ 17. TESTES

- └─ unittest
- └─ pytest
- └─ TDD

└─ 18. FERRAMENTAS

PROFISSIONAIS

- └─ Git
- └─ Docker
- └─ virtualenv
- └─ pip

└─ 19. ALGORITMOS E

PERFORMANCE

- └─ complexidade
- └─ Big-O
- └─ otimização
- └─ estruturas eficientes